

NetGuard: Network Vulnerability Scanner

Design & Technical Specification

Author: Aiman Nurzharfan

Date: December 2025

Version: 1.2

1. Executive Summary

NetGuard is a custom-developed, multi-threaded network reconnaissance tool designed to automate the identification of active network services. Bridging the gap between manual connectivity testing and heavyweight vulnerability scanners, NetGuard provides a lightweight, Python-based solution for Network Administrators to audit port states and service versions rapidly.

2. Introduction

The application is built on a modular architecture comprising four distinct subsystems:

2.1 Input Validation Module

- **Function:** Accepts and sanitizes user input from the Command Line Interface (CLI).
- **Logic:** Utilizes `socket.inet_aton` to strictly validate IPv4 address formats, preventing execution errors from malformed inputs.

2.2 Concurrent Scanning Engine

- **Core Technology:** `concurrent.futures.ThreadPoolExecutor`
- **Logic:** Instead of sequential processing, the engine spawns a pool of **100 concurrent worker threads**. Each thread manages a raw TCP socket connection.
- **Connection Mechanism:** Uses `socket.connect_ex` to perform a non-blocking TCP handshake. A return code of `0` confirms an **OPEN** state; all other codes are interpreted as **CLOSED** or **FILTERED**.
- **Timeout Handling:** A strict 1.0-second timeout is enforced per socket to prevent the scanner from hanging on firewall-dropped packets.

2.3 Service Enumeration (Banner Grabbing)

- **Function:** Identifies the application running on an open port.
- **Logic:** Upon a successful connection, the scanner listens for a "Welcome Banner" (first 1024 bytes). This data is decoded from bytes to UTF-8, sanitized of newline characters, and truncated to 200 characters for display.

2.4 Automated Reporting

- **Core Technology:** `fpdf2` Library.
- **Output:** Generates a standardized PDF report (`netguard_last_scan.pdf`) containing:
 - Scan Metadata (Target IP, Start/End Time, Duration).
 - Vulnerability Table (Port Number, Service Banner).
- **Persistence:** The system is designed to overwrite the previous report file on every execution, ensuring the "Last Scan" is always the most current data.

3. Technical Stack

Language: Python 3.10+

Dependencies:

- `colorama`: For ANSI color-coded terminal output (UX).
- `fpdf2`: For programmatic PDF generation.
- `threading` & `socket`: Core Python standard libraries.

4. Security & Compliance

This tool is engineered for defensive security (Blue Team) operations. It includes explicit disclaimers regarding authorization. The architectural design focuses on "Passive Reconnaissance" (Banner Grabbing) rather than "Active Exploitation," keeping it compliant with standard ethical hacking guidelines.